

The PNP–BV Mathematical Model

from mass conservation to the residual our solver inverts

Onboarding doc 02 of 06

May 11, 2026

What this document is for

Builds the equations of the forward solver from scratch. Assumes you remember what a partial derivative is and have done multivariable calculus and ODEs; assumes you have *not* taken a PDE course. By the end you should be able to read the file `Forward/bv_solver/forms_logc_muh.py` and recognise every term as something with a physical name.

Contents

1	Why we need PDEs at all	1
1.1	Steady state	1
2	Building the PDE system one piece at a time	2
2.1	Mass conservation	2
2.2	The flux: Nernst–Planck	2
2.3	The potential: Poisson	2
2.4	Putting it together: the PNP system	2
3	Boundary conditions	3
3.1	At the bulk side $x = L_{\text{eff}}$ (easy)	3
3.2	At the electrode side $x = 0$ (the interesting one)	3
4	The Bikerman closure for multi-ion electrolytes	3
5	Choosing the primary variables wisely	4
5.1	Why log-concentration ($u_i = \ln c_i$)	4
5.2	Why μ for protons (the <code>logc_muh</code> formulation)	4
6	Nondimensionalization	5
7	Phase 6α and 6β extensions	5
7.1	Phase 6α : water self-ionization	5
7.2	Phase 6β : cation hydrolysis at the OHP	5
8	From PDE to a residual: a quick FEM tour	6
8.1	Weak form	6
8.2	Newton	6

1 Why we need PDEs at all

The simplest model of an electrochemical cell would be a single **stirred tank**: pretend the electrolyte has uniform concentrations everywhere, the surface chemistry happens at one “rate”, and you write down a system of ODEs in time. Many ORR papers do this.

The problem: the chemistry happens in a region (the electric double layer + the diffusion layer) that is at most $\sim 100 \mu\text{m}$ thick, inside which concentrations and the electric potential vary by *many orders of magnitude*. Local pH near the surface can swing from the bulk value of 4 up to almost 10; the electric potential ϕ can drop by 1 V over a Debye length of less than 1 nm. Treating any of those as “uniform” throws away the physics.

So we keep $c_i(x, t)$ and $\phi(x, t)$ as functions of position $x \in [0, L_{\text{eff}}]$ and time t . Mass conservation and Gauss’s law give us coupled **partial differential equations**: derivatives in both x and t .

1.1 Steady state

For analyzing a single V_{RHE} point in a current-voltage curve we hold the applied voltage fixed for long enough that $\partial_t c_i = 0$. So in practice the residual we solve is the *steady-state* system. The time-dependent form is in the `forms_logc_muh.py _build_residual` routine but the time derivative gets multiplied by an artificial relaxation parameter that we drive to zero (this is sometimes called “pseudo-time stepping” and helps Newton’s method).

2 Building the PDE system one piece at a time

2.1 Mass conservation

For each chemical species i in the electrolyte, write down its concentration $c_i(x, t)$ in mol/m^3 and its molar flux $\mathbf{J}_i(x, t)$ in $\text{mol}/\text{m}^2/\text{s}$. Conservation of moles gives **the continuity equation**:

$$\frac{\partial c_i}{\partial t} + \nabla \cdot \mathbf{J}_i = R_i, \quad (1)$$

where R_i is a volumetric source/sink (units $\text{mol}/\text{m}^3/\text{s}$) due to homogeneous chemical reactions. For most of our species $R_i = 0$ because we lump all chemistry into a boundary condition at the electrode. Phase 6 α added a non-zero R_{H^+} from water self-ionization and Phase 6 β adds a boundary source from cation hydrolysis at the OHP.

2.2 The flux: Nernst–Planck

The Nernst–Planck equation (doc 01 §6.1):

$$\mathbf{J}_i = -D_i \nabla c_i - \frac{z_i F D_i}{RT} c_i \nabla \phi + c_i \mathbf{v}. \quad (2)$$

The three terms are diffusion (down the concentration gradient), migration (charged ions drift along the electric field), and convection (carried by the bulk velocity field). In our 1D slab the convection term is folded into the choice of L_{eff} — see doc 01 §6.3 — so the operative flux is just the first two terms.

2.3 The potential: Poisson

In a continuous medium, Gauss's law of electrostatics is

$$-\nabla \cdot (\varepsilon \nabla \phi) = \rho, \quad (3)$$

where ε is the dielectric permittivity of the solvent (for water at 25°C, $\varepsilon = 80 \varepsilon_0$) and ρ is the local volumetric charge density. The charge density is just the sum of ion charges:

$$\rho = F \sum_i z_i c_i. \quad (4)$$

(3) and (4) close the system: ϕ is determined by the ion distribution, and the ion distribution is shaped by ϕ through (2). This nonlinear feedback is the heart of EDL physics.

2.4 Putting it together: the PNP system

$$\boxed{\begin{aligned} \frac{\partial c_i}{\partial t} + \nabla \cdot \left(-D_i \nabla c_i - \frac{z_i F D_i}{RT} c_i \nabla \phi \right) &= R_i \quad \text{for each species } i \\ -\nabla \cdot (\varepsilon \nabla \phi) &= F \sum_i z_i c_i \end{aligned}} \quad (5)$$

$$(6)$$

This is a coupled, nonlinear, parabolic–elliptic system. In 1D steady state on $x \in (0, L_{\text{eff}})$ it becomes a coupled boundary-value problem.

3 Boundary conditions

3.1 At the bulk side $x = L_{\text{eff}}$ (easy)

Here we are far enough from the electrode that the EDL has fully relaxed and concentrations recover their bulk values. Dirichlet:

$$c_i(L_{\text{eff}}) = c_i^{\text{bulk}}, \quad \phi(L_{\text{eff}}) = 0.$$

The choice of $\phi = 0$ at the bulk is just a gauge: we measure electric potential relative to the bulk solution.

3.2 At the electrode side $x = 0$ (the interesting one)

Three conditions hold simultaneously here:

(a) Faradaic flux from BV. The flux of each species into the boundary equals its stoichiometric draw in the surface reactions. For a reaction $\sum_p \nu_p S_p + n_e e^- \rightarrow \sum_q \nu_q S_q$ proceeding at rate r [mol/m²/s]:

$$\mathbf{J}_i \cdot \hat{\mathbf{n}}|_{x=0} = \sum_{\text{rxns}} \nu_i r, \quad (7)$$

where $\hat{\mathbf{n}}$ is the outward normal (pointing into the bulk). r is the Butler–Volmer rate (the BV equation in doc 01, or its log-rate form). For us, summing over R_{2e} and R_{4e} in parallel, the boundary flux of O₂ is $\nu_{\text{O}_2}^{(2e)} r_{2e} + \nu_{\text{O}_2}^{(4e)} r_{4e}$ with $\nu_{\text{O}_2} = -1$ for both reactions.

(b) The Stern compact layer. The metal-side surface charge σ_{metal} is set by the rigid Stern capacitor as

$$\sigma_{\text{metal}} = C_S (\phi_m - \phi(0)) = C_S \psi_S, \quad (8)$$

where ϕ_m is the metal potential, $\phi(0)$ is the diffuse-side solution potential at the OHP, and ψ_S is the **Stern voltage drop**. Continuity of the electric displacement gives a Robin condition for ϕ at $x = 0$:

$$-\varepsilon \frac{\partial \phi}{\partial x} \Big|_{x=0} = -\sigma_{\text{metal}} = -C_S (\phi_m - \phi(0)). \quad (9)$$

The applied potential V enters through ϕ_m via $\phi_m = V - V_{\text{ref}}^\circ$ for whatever reference is chosen.

(c) Equilibrium for non-reactive species. For ions that do not react at the surface (e.g. SO_4^{2-} , K^+ in our baseline), the flux into the boundary is zero: $\mathbf{J}_i \cdot \hat{\mathbf{n}}|_{x=0} = 0$. In our **analytic Bikerman counter-ion** treatment we go further: we *don't even keep* a NP equation for these ions, but instead substitute their equilibrium Boltzmann profile (Bikerman form, doc 01) directly. They act as a closed-form contribution to the charge density in (4). This saves a degree of freedom per species and is exact at steady state for non-reactive ions.

4 The Bikerman closure for multi-ion electrolytes

For non-reactive analytic counter-ions we use a **multi-ion shared- θ Bikerman closure**. In the single-ion form it reduced to the standard Bikerman expression (doc 01). The multi-ion form that the codebase implements (in `Forward/bv_solver/boltzmann.py`) is:

$$c_i(x) = \frac{c_i^{\text{bulk}} \exp(-z_i \phi(x)/V_T) (1 - A_{\text{dyn}}(x))}{\theta_b + \sum_j a_j c_j^{\text{bulk}} \exp(-z_j \phi(x)/V_T)}, \quad (10)$$

where a_i is the molar volume of species i , θ_b is the bulk packing fraction, and $A_{\text{dyn}}(x)$ is the local volume fraction occupied by the dynamic species (the ones for which we still solve a NP equation). This makes the steric saturation a *shared* budget: the dynamic species crowding takes up part of the available volume; the analytic Bikerman ions split what's left according to their bulk concentrations and their Boltzmann factors.

The single hard rule about this:

Hard rule (don't violate)

The IC seed and the residual must agree about steric saturation. The initializer (`set_initial_conditions_debye_boltzmann_*`) seeds the composite- ψ matched-asymptotic with multi-species γ ; the residual side calls `build_steric_boltzmann_expressions`. Mixing a Bikerman IC with an ideal-counter-ion residual (or vice versa) cold-fails on the saturated manifold. This is documented as “Hard rule #3” in the project `CLAUDE.md`.

5 Choosing the primary variables wisely

5.1 Why log-concentration ($u_i = \ln c_i$)

Concentrations in our problem range from 10^{-12} mol/m³ (deep-cathodic H^+ depletion) up to $\sim 10^4$ mol/m³ (close-packed counter-ion) — 16 orders of magnitude. Solving for c_i directly causes Newton

iterates to step into negative concentrations and crash. Instead we change variable to

$$u_i = \ln c_i, \quad c_i = e^{u_i}, \quad (11)$$

which automatically keeps $c_i > 0$ no matter what Newton step is taken. The diffusion and migration terms (2) become:

$$\mathbf{J}_i = -D_i c_i \nabla u_i - \frac{z_i F D_i}{RT} c_i \nabla \phi = -D_i c_i \nabla \left(u_i + \frac{z_i F}{RT} \phi \right).$$

Notice the bracketed combination $u_i + (z_i F/RT)\phi$ is exactly the dimensionless electrochemical potential $\tilde{\mu}_i/RT$ minus its standard-state piece. This is the cue for the next step.

5.2 Why μ for protons (the `logc_muh` formulation)

Even with $u_i = \ln c_i$, the proton variable u_H and the potential ϕ each vary by tens of V_T across the diffuse layer, while their *sum* (the electrochemical potential μ_H) varies much more gently. The combination

$$\mu_H = u_H + e_m z_H \phi / V_T \quad (z_H = +1)$$

(where e_m is the dimensionless electromigration prefactor, equal to 1 in physical units) becomes our new primary variable. Then u_H is recovered as $u_H = \mu_H - z_H \phi / V_T$. Newton iterates on μ_H are smooth where iterates on u_H would oscillate.

This is the meaning of the formulation tag `formulation = "logc_muh"`. The other species (O_2 , H_2O_2) stay in plain $u_i = \ln c_i$ since they are uncharged ($z = 0$) and their behaviour is dominated by diffusion, not migration.

Practical bookkeeping. The dispatcher in `Forward/bv_solver/dispatch.py` routes the residual builder to either `forms_logc.py` (all species in $\ln c$) or `forms_logc_muh.py` (proton in μ_H , others in $\ln c$). Production work is on `forms_logc_muh.py`; the legacy `forms_logc.py` is kept for cross-check.

6 Nondimensionalization

The solver works in scaled (“hatted”) variables to keep matrix conditioning sane. The scales are:

Quantity	Scale
position x	$\hat{x} = x / L_{\text{eff}}$
concentration c_i	$\hat{c}_i = c_i / c_{\text{ref}}$ where $c_{\text{ref}} = \text{bulk O}_2$ typically
potential ϕ	$\hat{\phi} = \phi / V_T$
time t	$\hat{t} = t \cdot D_{\text{ref}} / L_{\text{eff}}^2$
rate r	$\hat{r} = r \cdot L_{\text{eff}} / (D_{\text{ref}} c_{\text{ref}})$
k_0	rate constant scaled the same way as r

Two dimensionless numbers fall out of the scaling:

- The **electromigration prefactor** $e_m = 1$ (it goes away in the natural scaling but is kept as a knob for diagnostic use).
- The **Debye number** $\hat{\lambda}_D^2 = \varepsilon V_T / (F^2 c_{\text{ref}} L_{\text{eff}}^2)$, which is tiny ($\sim 10^{-10}$). It multiplies the Poisson Laplacian, so the EDL is a *singular perturbation* layer of width $\hat{\lambda}_D \sim 10^{-5}$ in scaled units. The mesh has to resolve this layer; doc 03 explains how.

The constants live in `Nondim/` (scaling transforms + physical constants) and `scripts/_bv_common.py` (production constants like `V_T`, `C_O2_BULK_HAT`, `KO_HAT_R2E`, etc.). Cross-check any dimensional formula by computing in SI and then dividing by the scales above.

7 Phase 6 α and 6 β extensions

The above is the production stack as of Phase 5 γ . Two extensions are layered on top.

7.1 Phase 6 α : water self-ionization

Adds the homogeneous reaction $\text{H}_2\text{O} \rightleftharpoons \text{H}^+ + \text{OH}^-$ with equilibrium $K_w = c_{\text{H}^+} c_{\text{OH}^-}$. Because OH^- is not a tracked species, we close it algebraically: $c_{\text{OH}^-}(x) = K_w / c_{\text{H}^+}(x)$. The H^+ source becomes

$$R_{\text{H}^+}(x) = k_w^{\text{eff}} (1 - c_{\text{H}^+}(x) c_{\text{OH}^-}(x) / K_w)$$

which vanishes at equilibrium. The flag is `enable_water_ionization=True`; default is off and the default-off code path is byte-equivalent to pre-6 α .

7.2 Phase 6 β : cation hydrolysis at the OHP

Adds a coverage variable $\Gamma_{\text{MOH}}(t)$ at the OHP satisfying:

$$\dot{\Gamma}_{\text{MOH}} = k_{\text{hyd}} (1 - \Gamma_{\text{MOH}} / \Gamma_{\text{max}}) c_M(0) - k_{\text{des}} \Gamma_{\text{MOH}}, \quad (12)$$

where:

- $c_M(0)$ is the cation concentration at the OHP from the Bikerman closure;
- k_{hyd} depends on the local field through Singh's field-dependent pK_a (Singh 2016 formula, doc 01);
- the Langmuir factor $(1 - \Gamma / \Gamma_{\text{max}})$ caps the coverage at one monolayer; this was added in v10a after v9 converged at ~ 6 monolayers (unphysical);
- k_{des} is the desorption rate (the inverse step).

Each forward step of Γ_{MOH} injects a proton flux into the boundary condition for H^+ , modifying the BV reaction rates indirectly. The whole thing is solved as an outer Picard loop around the inner Newton solve for the PNP+BV residual.

The relevant code is in `Forward/bv_solver/cation_hydrolysis.py`. Calibration of Γ_{max} , k_{hyd} , k_{des} to literature values is the v10b work documented in `docs/phase6/v10b_calibration_summary.md`.

8 From PDE to a residual: a quick FEM tour

Doc 03 covers this in detail; here is the quick version so you can read `forms_logc_muh.py` without panic.

8.1 Weak form

You cannot solve PDEs exactly on a computer. The **finite-element method (FEM)** chooses a finite-dimensional function space V_h (e.g. piecewise-linear functions on a mesh) and demands that the residual of the PDE be *orthogonal to* every test function $v \in V_h$:

$$\mathcal{R}(c_h, \phi_h)[v] = 0 \quad \forall v \in V_h.$$

For a NP equation like $\partial_t c + \nabla \cdot \mathbf{J} = R$ on the domain Ω , you multiply by a test function v , integrate over Ω , integrate the divergence by parts, and pull out boundary terms:

$$\int_{\Omega} \partial_t c v \, dx - \int_{\Omega} \mathbf{J} \cdot \nabla v \, dx + \int_{\partial\Omega} (\mathbf{J} \cdot \hat{\mathbf{n}}) v \, ds - \int_{\Omega} R v \, dx = 0.$$

The boundary integral is exactly where the Butler–Volmer flux (7) and the Stern term (9) enter. Firedrake (the FEM library we use) lets you write each of these integrals as one Python expression on a UFL form (`F = inner(J, grad(v))*dx + bv_flux*v*ds(electrode)`, and so on); see `forms_logc_muh.py` lines ~200–400.

8.2 Newton

After discretization, the residual becomes a vector function $F(U)$ of the global degrees-of-freedom vector U (one entry per mesh node per primary variable). We solve $F(U) = 0$ by Newton’s method:

$$J(U^k) \Delta U = -F(U^k), \quad U^{k+1} = U^k + \Delta U,$$

with $J = \partial F / \partial U$ assembled symbolically by Firedrake and the linear solve done by PETSc. You have seen Newton in numerical methods; the only twist here is that J is a sparse matrix of size $\sim 10^4$ – 10^5 , and that F is so nonlinear that Newton from cold-start frequently fails. Doc 03 covers the **continuation** tricks that get around this.

9 What goes in, what comes out

Inputs to the solver.

- Geometry: L_{eff} , mesh $N_x \times N_y$, grading factor for the EDL refinement.
- Bulk concentrations c_i^{bulk} .
- Reaction set: $\{E_r^\circ, k_0^{(r)}, \alpha^{(r)}, n_e^{(r)}\}$ for $r \in \{2e, 4e\}$.
- Stern capacitance C_S .
- Counterion list (Bikerman entries with charge, molar volume, and bulk concentration).
- Applied voltage V_{RHE} (often a list, swept).

Outputs.

- Per voltage point: convergence flag, total disk current i_D , peroxide current i_P , surface concentrations $c_i(0)$, surface electric potential $\phi(0)$, Γ_{MOH} coverage, BV-rate diagnostics.
- A JSON dump under `StudyResults/<study_name>/`.
- Plots, when the driver script generates them.